

DSA lab exercise with the objective, problem statement, menu-driven C program covering the major doubly linked-list operations, and an explanation of the node-table display.

DSA Lab Exercise: Doubly Linked List

Lab No. ____

Title

Implementation of Doubly Linked List and Its Operations

Objective

To write and execute a **menu-driven C program** to implement a doubly linked list and perform the following operations:

1. Create a doubly linked list.
2. Insert a node at the beginning.
3. Insert a node at the end.
4. Insert a node after a given node.
5. Insert a node before a given node.
6. Delete a node from the beginning.
7. Delete a node from the end.
8. Delete a node with a given value.
9. Search for an element.
10. Display the list in forward direction.
11. Display the list in backward direction.
12. Display each node with its `prev`, `data`, and `next` fields.

Problem Statement

Write a **menu-driven C program** to implement a doubly linked list using dynamic memory allocation.

Each node of the doubly linked list should contain three fields:

```
+-----+-----+-----+
|      prev      |  data  |      next      |
+-----+-----+-----+
```

The `prev` field stores the address of the previous node, `data` stores the value, and `next` stores the address of the next node.

The program should allow the user to perform all the operations through a menu.

The **Display Node Table** option should display the list in the following format:

	PREV ADDRESS	DATA	NEXT ADDRESS

	NULL	10	0x1234
	0x1230	20	0x1250
	0x1234	30	0x1270
	0x1250	40	NULL

The actual memory addresses will be different each time the program runs.

C Program

C

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    struct Node *prev;
    int data;
    struct Node *next;
};

struct Node *head = NULL;

/*-----
   Function to create a new node
   -----*/
struct Node* createNode(int value)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->prev = NULL;
    newNode->data = value;
```

```

        newNode->next = NULL;

        return newNode;
    }

    /*-----
       Insert at Beginning
    -----*/
void insertBeginning(int value)
{
    struct Node *newNode;

    newNode = createNode(value);

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }

    printf("\n%d inserted at beginning.\n", value);
}

/*-----
   Insert at End
-----*/
void insertEnd(int value)
{
    struct Node *newNode;
    struct Node *temp;

    newNode = createNode(value);

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        temp = head;

        while (temp->next != NULL)
        {
            temp = temp->next;
        }
    }
}

```

```

        temp->next = newNode;
        newNode->prev = temp;
    }

    printf("\n%d inserted at end.\n", value);
}

/*-----
   Insert After a Given Value
   -----*/
void insertAfter(int searchValue, int value)
{
    struct Node *temp;
    struct Node *newNode;

    temp = head;

    while (temp != NULL && temp->data != searchValue)
    {
        temp = temp->next;
    }

    if (temp == NULL)
    {
        printf("\n%d not found in the list.\n", searchValue);
        return;
    }

    newNode = createNode(value);

    newNode->prev = temp;
    newNode->next = temp->next;

    if (temp->next != NULL)
    {
        temp->next->prev = newNode;
    }

    temp->next = newNode;

    printf("\n%d inserted after %d.\n", value, searchValue);
}

/*-----
   Insert Before a Given Value
   -----*/
void insertBefore(int searchValue, int value)
{
    struct Node *temp;

```

```

    struct Node *newNode;

    temp = head;

    while (temp != NULL && temp->data != searchValue)
    {
        temp = temp->next;
    }

    if (temp == NULL)
    {
        printf("\n%d not found in the list.\n", searchValue);
        return;
    }

    newNode = createNode(value);

    newNode->next = temp;
    newNode->prev = temp->prev;

    if (temp->prev != NULL)
    {
        temp->prev->next = newNode;
    }
    else
    {
        head = newNode;
    }

    temp->prev = newNode;

    printf("\n%d inserted before %d.\n", value, searchValue);
}

/*-----
   Delete from Beginning
   -----*/
void deleteBeginning()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("\nList is empty.\n");
        return;
    }

    temp = head;
    head = head->next;

    if (head != NULL)

```

```

    {
        head->prev = NULL;
    }

    printf("\n%d deleted from beginning.\n", temp->data);

    free(temp);
}

```

```

/*-----
Delete from End
-----*/

```

```

void deleteEnd()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("\nList is empty.\n");
        return;
    }

    temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    if (temp->prev == NULL)
    {
        head = NULL;
    }
    else
    {
        temp->prev->next = NULL;
    }

    printf("\n%d deleted from end.\n", temp->data);

    free(temp);
}

```

```

/*-----
Delete a Given Value
-----*/

```

```

void deleteValue(int value)
{
    struct Node *temp;

```

```

temp = head;

while (temp != NULL && temp->data != value)
{
    temp = temp->next;
}

if (temp == NULL)
{
    printf("\n%d not found in the list.\n", value);
    return;
}

/* If deleting the first node */
if (temp->prev == NULL)
{
    head = temp->next;
}
else
{
    temp->prev->next = temp->next;
}

/* If deleting a node other than last */
if (temp->next != NULL)
{
    temp->next->prev = temp->prev;
}

printf("\n%d deleted from the list.\n", value);

free(temp);
}

/*-----
Search an Element
-----*/
void search(int value)
{
    struct Node *temp;
    int position = 1;

    temp = head;

    while (temp != NULL)
    {
        if (temp->data == value)
        {
            printf("\n%d found at position %d.\n",
                value, position);
            return;

```

```

    }

    temp = temp->next;
    position++;
}

printf("\n%d not found in the list.\n", value);
}

/*-----
   Display Forward
   -----*/
void displayForward()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("\nList is empty.\n");
        return;
    }

    temp = head;

    printf("\nForward List:\n");

    while (temp != NULL)
    {
        printf("%d", temp->data);

        if (temp->next != NULL)
        {
            printf(" <-> ");
        }

        temp = temp->next;
    }

    printf(" -> NULL\n");
}

/*-----
   Display Backward
   -----*/
void displayBackward()
{
    struct Node *temp;

    if (head == NULL)
    {

```

```

        printf("\nList is empty.\n");
        return;
    }

    temp = head;

    /* Move to the last node */
    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    printf("\nBackward List:\n");

    while (temp != NULL)
    {
        printf("%d", temp->data);

        if (temp->prev != NULL)
        {
            printf(" <-> ");
        }

        temp = temp->prev;
    }

    printf(" -> NULL\n");
}

/*-----
   Display Node Table
   -----*/
void displayTable()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("\nList is empty.\n");
        return;
    }

    temp = head;

    printf("\n");
    printf("Doubly Linked List Node Table\n");

    printf("-----\n");
    printf("| %20s | %10s | %20s |\n",
           "PREV ADDRESS",
           "DATA",

```

```

        "NEXT ADDRESS");

printf("-----\n");

while (temp != NULL)
{
    printf("| %20p | %10d | %20p |\n",
           (void*)temp->prev,
           temp->data,
           (void*)temp->next);

    temp = temp->next;
}

printf("-----\n");
}

/*-----
   Count Nodes
-----*/
void countNodes()
{
    struct Node *temp;
    int count = 0;

    temp = head;

    while (temp != NULL)
    {
        count++;
        temp = temp->next;
    }

    printf("\nNumber of nodes = %d\n", count);
}

/*-----
   Delete Complete List
-----*/
void deleteList()
{
    struct Node *temp;

    while (head != NULL)
    {
        temp = head;
        head = head->next;
        free(temp);
    }

    printf("\nComplete list deleted.\n");
}

```

```

}

/*-----
Main Function
-----*/

int main()
{
    int choice;
    int value;
    int searchValue;

    do
    {
        printf("\n\n");
        printf("=====\n");
        printf("          DOUBLY LINKED LIST MENU\n");
        printf("=====\n");

        printf("1.  Insert at Beginning\n");
        printf("2.  Insert at End\n");
        printf("3.  Insert After a Given Value\n");
        printf("4.  Insert Before a Given Value\n");
        printf("5.  Delete from Beginning\n");
        printf("6.  Delete from End\n");
        printf("7.  Delete a Given Value\n");
        printf("8.  Search an Element\n");
        printf("9.  Display Forward\n");
        printf("10. Display Backward\n");
        printf("11. Display Node Table\n");
        printf("12. Count Nodes\n");
        printf("13. Delete Complete List\n");
        printf("14. Exit\n");

        printf("=====\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:

                printf("Enter value: ");
                scanf("%d", &value);

                insertBeginning(value);
                break;

            case 2:

```

```
printf("Enter value: ");
scanf("%d", &value);

insertEnd(value);
break;

case 3:

printf("Enter value after which to insert: ");
scanf("%d", &searchValue);

printf("Enter new value: ");
scanf("%d", &value);

insertAfter(searchValue, value);
break;

case 4:

printf("Enter value before which to insert: ");
scanf("%d", &searchValue);

printf("Enter new value: ");
scanf("%d", &value);

insertBefore(searchValue, value);
break;

case 5:

deleteBeginning();
break;

case 6:

deleteEnd();
break;

case 7:

printf("Enter value to delete: ");
scanf("%d", &value);

deleteValue(value);
break;
```

```

        case 8:

            printf("Enter value to search: ");
            scanf("%d", &value);

            search(value);
            break;

        case 9:

            displayForward();
            break;

        case 10:

            displayBackward();
            break;

        case 11:

            displayTable();
            break;

        case 12:

            countNodes();
            break;

        case 13:

            deleteList();
            break;

        case 14:

            deleteList();
            printf("\nProgram terminated.\n");
            break;

        default:

            printf("\nInvalid choice! Please try again.\n");

    }

```

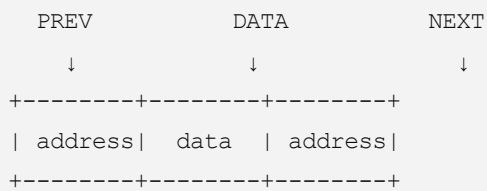
Explanation of the Program

Node Structure

C

```
struct Node
{
    struct Node *prev;
    int data;
    struct Node *next;
};
```

A doubly linked-list node contains:



For example:

```
NULL <- [10] <-> [20] <-> [30] -> NULL
```

For node 20 :

```
prev → address of node 10
data → 20
next → address of node 30
```

Important Operations

Operation	Function
Insert beginning	<code>insertBeginning()</code>
Insert end	<code>insertEnd()</code>
Insert after	<code>insertAfter()</code>
Insert before	<code>insertBefore()</code>
Delete beginning	<code>deleteBeginning()</code>
Delete end	<code>deleteEnd()</code>
Delete value	<code>deleteValue()</code>

Operation	Function
Search	<code>search()</code>
Forward display	<code>displayForward()</code>
Backward display	<code>displayBackward()</code>
Node table	<code>displayTable()</code>
Count nodes	<code>countNodes()</code>
Delete entire list	<code>deleteList()</code>

Example

After inserting:

```
10, 20, 30, 40
```

Forward display:

```
10 <-> 20 <-> 30 <-> 40 -> NULL
```

Backward display:

```
40 <-> 30 <-> 20 <-> 10 -> NULL
```

Node table:

PREV ADDRESS	DATA	NEXT ADDRESS
(nil)	10	0x....
0x....	20	0x....
0x....	30	0x....
0x....	40	(nil)

The table is particularly useful for teaching students the **memory representation** of a doubly linked list.

Complexity Analysis

Operation	Complexity
Insert at beginning	O(1)
Insert at end	O(n)
Insert before/after a value	O(n)
Delete beginning	O(1)
Delete end	O(n)
Delete a value	O(n)
Search	O(n)
Forward traversal	O(n)
Backward traversal	O(n)
Display node table	O(n)
Count nodes	O(n)
Delete complete list	O(n)

Lab Task for Students

Test the program using the following sequence:

1. Insert 10 , 20 , 30 , 40 , 50 at the end.
2. Display the list forward.
3. Display the list backward.
4. Display the node table.
5. Insert 25 after 20 .
6. Insert 5 before 10 .
7. Delete the first node.
8. Delete the last node.
9. Delete node 30 .
10. Search for 40 .
11. Display the final node table.
12. Count the number of nodes.
13. Delete the complete list.

This gives students practice with **creation, insertion, deletion, searching, traversal, memory addresses, and complexity analysis** in one lab exercise.